

```

import pandas as pd
import lightgbm as lgb

# =====
# 1. โหลดข้อมูลเฉพาะส่วนที่ใช้สอนโมเดล (Train & Validation)
# =====
print("📂 กำลังโหลดข้อมูล Train และ Validation...")
train_df = pd.read_csv('Train_Data_70.csv')
val_df = pd.read_csv('Val_Data_15.csv')

# =====
# 2. จัดเตรียมข้อมูล (แยก Features และ Target)
# =====
# ระบุคอลัมน์ที่ "ไม่ใช่" เซนเซอร์ เพื่อตัดออก
drop_cols = ['timestamp', 'segment', 'train', 'anomaly']

# แยกข้อมูลและเจตนาสำหรับ Train
X_train = train_df.drop(columns=drop_cols)
y_train = train_df['anomaly']

# แยกข้อมูลและเจตนาสำหรับ Validation (ใช้คุม Early Stopping)
X_val = val_df.drop(columns=drop_cols)
y_val = val_df['anomaly']

# แปลงเป็น Dataset format ของ LightGBM เพื่อให้รันได้เร็วที่สุด
train_data = lgb.Dataset(X_train, label=y_train)
val_data = lgb.Dataset(X_val, label=y_val, reference=train_data)

# =====
# 3. ตั้งค่าสมองของ AI (Hyperparameters)
# =====
params = {
    'objective': 'binary',          # ปัญหา 2 คลาส (0 หรือ 1)
    'metric': 'auc',               # ใช้ AUC เป็นเกณฑ์วัด
    'boosting_type': 'gbdt',       # อัลกอริทึม Gradient Boosting
    'learning_rate': 0.05,         # ความเร็วในการเรียนรู้
    'num_leaves': 31,              # จำนวนใบไม้สูงสุด (ความซับซ้อนของกิ่ง)
    'feature_fraction': 0.8,       # สุ่มใช้เซนเซอร์ 80% ต่อต้น
    'random_state': 42,            # ล็อคค่าสุ่ม
    'verbose': -1                  # ปิดข้อความแจ้งเตือนที่ไม่จำเป็น
}

# =====
# 4. เริ่มฝึกฝนโมเดล (Training & Early Stopping)
# =====
print("🚀 เริ่มต้นเทรนโมเดล LightGBM...")

model = lgb.train(
    params,
    train_data,
    num_boost_round=1000,          # ให้สร้างต้นไม้สูงสุด 1,000 ต้น
    valid_sets=[train_data, val_data], # ใส่ข้อมูลให้ระบบตรวจสอบ
    valid_names=['Train', 'Validation'],
    callbacks=[
        lgb.early_stopping(stopping_rounds=50), # ถ้า Validation ไม่ดีขึ้น 50 ต้น ให้หยุด
        lgb.log_evaluation(period=50)           # รายงานผลทุกๆ 50 ต้น
    ]
)

print(f"🎉 เทรนเสร็จสิ้น! โมเดลหยุดที่ต้นไม้ต้นที่: {model.best_iteration}")

# =====
# 5. บันทึกโมเดลเก็บไว้ใช้งาน (Save Model)
# =====
model_filename = 'LightGBM_Anomaly_Model.txt'
model.save_model(model_filename)
print(f"💾 บันทึกโมเดลสำเร็จ! ไฟล์ชื่อ: '{model_filename}'")

```

📂 กำลังโหลดข้อมูล Train และ Validation...

🚀 เริ่มต้นเทรนโมเดล LightGBM...
 Training until validation scores don't improve for 50 rounds
 [50] Train's auc: 0.948933 Validation's auc: 0.520344
 [100] Train's auc: 0.981803 Validation's auc: 0.5399
 [150] Train's auc: 0.995104 Validation's auc: 0.537002
 Early stopping, best iteration is:
 [126] Train's auc: 0.990431 Validation's auc: 0.544399

✅ เทรนเสร็จสิ้น! โมเดลหยุดที่ต้นไม้ต้นที่: 126
 💾 บันทึกโมเดลสำเร็จ! ไฟล์ชื่อ: 'LightGBM_Anomaly_Model.txt'

```

import pandas as pd
import lightgbm as lgb
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    mean_squared_error,
    mean_absolute_error,
    confusion_matrix
)

# =====
# 1. โหลดโมเดลและข้อสอบ (Test Set)
# =====
print("📂 กำลังโหลดข้อสอบ (Test Set) และโมเดล...")
test_df = pd.read_csv('Test_Data_15.csv')

model = lgb.Booster(model_file='LightGBM_Anomaly_Model.txt')

drop_cols = ['timestamp', 'segment', 'train', 'anomaly']
X_test = test_df.drop(columns=drop_cols)
y_true = test_df['anomaly']

# =====
# 2. ให้โมเดลทำนาย (Predict)
# =====
print("🧠 โมเดลกำลังทำนายผล...")
y_pred_prob = model.predict(X_test)
y_pred_class = (y_pred_prob >= 0.5).astype(int)

# =====
# 3. คำนวณสถิติการทายถูก/ผิดแบบเจาะลึก (ตามคำขอ)
# =====
# ดึงค่าจากตาราง Confusion Matrix ออกมาเป็นตัวแปร 4 ตัว
# tn (True Negative) = ปกติจริง ทายว่าปกติ
# fp (False Positive) = ปกติจริง แต่ทายว่าพัง
# fn (False Negative) = พังจริง แต่ทายว่าปกติ
# tp (True Positive) = พังจริง ทายว่าพัง
tn, fp, fn, tp = confusion_matrix(y_true, y_pred_class).ravel()

# หาจำนวนความจริงที่มีในข้อสอบ
actual_nominals = tn + fp
actual_anomalies = fn + tp

# หาจำนวนที่ AI ทายถูกในแต่ละกลุ่ม
correct_nominals = tn
correct_anomalies = tp

# คำนวณเปอร์เซ็นต์ความแม่นยำรายกลุ่ม
pct_correct_nom = (correct_nominals / actual_nominals) * 100 if actual_nominals > 0 else 0
pct_correct_anom = (correct_anomalies / actual_anomalies) * 100 if actual_anomalies > 0 else 0

# =====
# 4. ประเมินผลและแสดงรายงาน (TEST METRICS REPORT)
# =====
print("\n" + "="*50)
print("📊 รายงานผลการทดสอบเจาะลึก (TEST METRICS REPORT)")
print("="*50)

print("🔴 [สรุปการทายผลรายกลุ่ม (Class-wise Accuracy)]")
print(f"✅ ข้อมูล 'ปกติ (0)' ทั้งหมดในข้อสอบ : {actual_nominals:,} บรรทัด")
print(f"→ AI ทายถูกต้อง : {correct_nominals:,} บรรทัด")
print(f"→ คิดเป็นความแม่นยำ : {pct_correct_nom:.2f}%")
print(f"{'-' * 40}")
print(f"❌ ข้อมูล 'พัง (1)' ทั้งหมดในข้อสอบ : {actual_anomalies:,} บรรทัด")
print(f"→ AI ทายถูกต้อง (จับผิดได้) : {correct_anomalies:,} บรรทัด")
print(f"→ คิดเป็นความแม่นยำ : {pct_correct_anom:.2f}% <-- (นี่คือค่า Recall ที่สำคัญที่สุด!)")

print("\n" + "="*50)
print("📈 [Metrics มาตรฐาน (Classification & Regression)]")

acc = accuracy_score(y_true, y_pred_class)
print(f"✅ Accuracy : {acc:.4f} (ทายถูกทั้งหมดทุกคลาส {acc*100:.2f}%)")

prec = precision_score(y_true, y_pred_class)
print(f"🔍 Precision : {prec:.4f} (เมื่อเตือนว่าพัง เชื่อถือได้ {prec*100:.2f}%)")

rec = recall_score(y_true, y_pred_class)
print(f"🔎 Recall : {rec:.4f} (ตรงกับความแม่นยำของกลุ่มพังด้านบน)")

f1 = f1_score(y_true, y_pred_class)
print(f"📊 F1 Score : {f1:.4f} (ความสมดุลระหว่าง Precision/Recall)")

```

```
mse = mean_squared_error(y_true, y_pred_prob)
mae = mean_absolute_error(y_true, y_pred_prob)
print(f"📊 MSE (ความคลาดเคลื่อนของความน่าจะเป็น): {mse:.4f}")
print(f"📊 MAE (ความคลาดเคลื่อนของความน่าจะเป็น): {mae:.4f}")
print("="*50)
```

```
print("\n📊 สรุปตาราง Confusion Matrix:")
print(f"[ทายปกติ] | [ทายพัง]")
print(f" {tn:>6} | {fp:>6} <-- ความจริงคือ ปกติ (0)")
print(f" {fn:>6} | {tp:>6} <-- ความจริงคือ พัง (1)")
```

🚦 กำลังโหลดข้อสอบ (Test Set) และโมเดล...

🧠 โมเดลกำลังทำนายผล...

📊 รายงานผลการทดสอบเจาะลึก (TEST METRICS REPORT)

🔴 [สรุปการทายผลรายกลุ่ม (Class-wise Accuracy)]

✅ ข้อมูล 'ปกติ (0)' ทั้งหมดในข้อสอบ : 9,151 บรรทัด

-> AI ทายถูกต้อง : 8,336 บรรทัด

-> คิดเป็นความแม่นยำ : 91.09%

❌ ข้อมูล 'พัง (1)' ทั้งหมดในข้อสอบ : 3,738 บรรทัด

-> AI ทายถูกต้อง (จับผิดได้) : 78 บรรทัด

-> คิดเป็นความแม่นยำ : 2.09% <-- (นี่คือค่า Recall ที่สำคัญที่สุด!)

📊 [Metrics มาตรฐาน (Classification & Regression)]

✓ Accuracy : 0.6528 (ทายถูกทั้งหมดทุกคลาส 65.28%)

🔍 Precision : 0.0873 (เมื่อเตือนว่าพัง เชื่อถือได้ 8.73%)

🔍 Recall : 0.0209 (ตรงกับความแม่นยำของกลุ่มพังด้านบน)

⚖️ F1 Score : 0.0337 (ความสมดุลระหว่าง Precision/Recall)

📊 MSE (ความคลาดเคลื่อนของความน่าจะเป็น): 0.2682

📊 MAE (ความคลาดเคลื่อนของความน่าจะเป็น): 0.4303

📊 สรุปตาราง Confusion Matrix:

[ทายปกติ]	[ทายพัง]	
8336	815	<-- ความจริงคือ ปกติ (0)
3660	78	<-- ความจริงคือ พัง (1)